

Dashboard API Contract

Every endpoint the dashboard calls, what each role may call, and the exact fields that come back — so the screens Paul builds and the server Chidera runs match, view by view.

How to use this. Each dashboard view below lists the endpoints it needs and the fields it reads. If a field here does not match the server, the server is wrong. If a screen needs something not listed, it goes in *Gaps* at the end, with an owner, rather than being worked around in the browser. The design these views follow is the shareable *Risk Radar Desk* page.

Rules both sides follow

- **One base path, same origin.** Everything is under /v1. In development the Vite server forwards /v1 to the API on port 8000.
- **Sign-in is a cookie.** POST /v1/auth/login sets a server-side session in a cookie JavaScript cannot read. Every request sends it (credentials: 'include'); nothing stores a token in the browser.
- **The menu comes from the server.** GET /v1/auth/me returns the role's permissions; Paul builds navigation and hides buttons from that list. The server still refuses anything not allowed (403), so hiding is convenience, not security.
- **Money is integer kobo.** Any field ending _minor is kobo as a whole number. Divide by 100 only for display; never add money as decimals.
- **Times are ISO 8601** with a time zone. occurred_at is when a payment happened; show that, not when it arrived.
- **Risk levels are words:** LOW, MEDIUM, HIGH, CRITICAL. The screen always shows the word as well as the colour.
- **Errors** return {"detail": ...}. 401: send to sign-in. 403: this role may not. 404: gone. 409: someone else got there first (for example two analysts taking the same case) — refetch and try again.
- **Advisory only.** No endpoint blocks, holds or reverses a payment, and no screen may suggest one does.

Who may call what

PERMISSION	ALLOWS	ANALYST	OPS LEAD	INFOSEC	ADMIN
cases:read	Worklist, cases, case detail, alerts, search, live stream	✓	✓	✓	—
cases:review	Take the next case, start review, add notes	✓	✓	✓	—
cases:set_outcome	Record fraud / false alarm / can't tell	✓	✓	—	—
cases:escalate	Escalate to InfoSec or Fraud Ops	✓	✓	✓	—
cases:close	Close a case (and close on disposition)	—	✓	—	—
cases:reassign	Assign a case to someone	—	✓	—	—
metrics:read	Operations and overview metrics	—	✓	✓	✓
admin:*	Rules, thresholds, lists, models, users	—	—	—	✓
audit:read	Read and verify the permanent record	—	—	—	✓

The administrator deliberately has no case permission: someone who can loosen a threshold must not be able to inspect the cases it missed. Two-step sign-in is mandatory for the lead and the administrator.

Signing in

ENDPOINT	SEND	GET BACK
POST /v1/auth/login	{ email, password, totp_code? }	{ status: "ok" "mfa_required", user? } – on "mfa_required", ask for the six-digit code and send again
GET /v1/auth/me	–	{ id, email, display_name, role, permissions[], mfa_satisfied }
POST /v1/auth/logout	–	ends the session on the server

View by view

Fraud analyst — Triage

WORKLIST · CASE PANEL · ONE-KEY OUTCOME · LIVE UPDATES

ENDPOINT	SEND	FIELDS THE SCREEN READS
GET /v1/worklist	scope = all mine unassigned breaching; risk_level?; limit ≤ 200 (default 60)	items[]: id, customer_name, risk_level, exposure_minor, attempted_minor, alert_count, distinct_beneficiaries, declined_count, new_device, channels[], signal_codes[], state, assignee_id, assignee_name, escalated_to, opened_at, last_alert_at, age_minutes, sla_state (OK DUE BREACHED), sla_remaining_minutes (negative = overdue), priority, recommendation { action, because, urgency: now soon routine, disposition_hint } summary: open_cases, total_exposure_minor, breaching, due_soon, unassigned, mine · total
POST /v1/worklist/next	–	{ case, resumed } – the highest-priority case, now assigned to the caller; 409 if taken in the same instant
GET /v1/cases/{id}	–	alerts[]: id, risk_level, amount_minor, channel, auth_result, occurred_at, signals[], features{12}, policy_trace[], attributions{}, rule_only_mode, model_name, model_version, ruleset_version, threshold_version, feature_spec_version timeline[]: occurred_at, amount_minor, channel, auth_result, alerted baseline[]: account_token, product_type, txn_30d, approved_value_30d_minor · notes[] · history[]
POST /v1/cases/{id}/disposition	{ outcome: CONFIRMED_FRAUD FALSE_POSITIVE INCONCLUSIVE, note?, close, followed_recommendation }	{ case_id, outcome, closed, steps[] } – closed is true only if the caller may close; otherwise the case waits for a lead. steps lists the state changes made
POST /v1/cases/{id}/review	–	take an unassigned case ("Take this case")
POST /v1/cases/{id}/notes	{ body } (1–4,000 characters)	the updated case
POST /v1/cases/{id}/escalate	{ target: INFOSEC FRAUD_OPS, note? }	the updated case
GET /v1/stream	server-sent events, cookie sign-in	event "alert": { alert_id, case_id, risk_level, score } → refetch the worklist event "alarm": { code, detail } → show a banner that stays until dismissed (for example the model is down and the system is on rules only)

Keys 1, 2, 3 send the three outcomes; N skips. Send followed_recommendation = (recommendation.disposition_hint === outcome). The stream reconnects on its own and replays what was missed, so no polling is needed.

Fraud operations lead — Is the desk coping?

CAPACITY CHART · BACKLOG · WAITING TIMES · TEAM · OUTCOMES

ENDPOINT	SEND	FIELDS THE SCREEN READS
GET /v1/metrics/operations	—	backlog[]: risk_level, cases, exposure_minor, avg_age_minutes · ageing[]: bucket (under 30m ... over 24h), cases · analysts[]: display_name, role, open_cases, closed_24h · outcomes[]: outcome, n · decided · false_positive_rate · sla_minutes { CRITICAL 15, HIGH 30, MEDIUM 240, LOW 1440 }
GET /v1/metrics/overview	hours = 1 ... 720 (default 24)	alert_volume[]: bucket (hour), risk_level, alerts · alert_budget { per_day, last_24h, utilisation } · transaction_volume[]: bucket, transactions, approved, declined · risk_mix[] · case_outcomes[] · signal_frequency[]: code, power, n · latency { p95_ms, scored, rule_only } · queue { depth, oldest_seconds } · active_model
POST /v1/cases/{id}/assign	{ assignee_id null }	the updated case
POST /v1/cases/{id}/close	—	the closed case (signs off an analyst's outcome)

The hourly capacity chart is alert_volume summed per hour against alert_budget.per_day ÷ 24. Bars over the line say so in words as well as colour.

Information security — Suspected break-ins

ESCALATIONS · NEW DEVICES · DORMANT ACCOUNTS

ENDPOINT	SEND	FIELDS THE SCREEN READS
GET /v1/worklist	scope = all	items where escalated_to = "INFOSEC", or new_device = true (filtered in the browser until gap 3 is closed)
GET /v1/cases	state = ESCALATED; risk_level?; channel?; sort = score recent opened; limit, offset	escalated cases, including ones no longer on the worklist
GET /v1/cases/{id}	—	features.days_since_account_activity (≥ 21 means dormant) and device_is_new_to_subject
POST /v1/cases/{id}/escalate · /notes	as above	InfoSec may investigate and escalate, but not record a fraud outcome (403)

Administrator — Rules, model and budget

NO CASE ACCESS, BY DESIGN

ENDPOINT	SEND	FIELDS THE SCREEN READS
GET · PATCH /v1/admin/rules[/{code}]	{ enabled?, params?, severity? }	the active ruleset's version, and each rule's code, power, severity, enabled, params – every change makes a new audited ruleset version
GET · POST /v1/admin/thresholds	{ p_monitor, p_review, p_hold, alert_min_level = MEDIUM, notes? }	the active set and its history
GET · POST · DELETE /v1/admin/lists	kind = SANCTIONED KNOWN_MULE ALLOWLIST; { kind, beneficiary_account_id, account_id?, note? }	list entries (tokens, never readable numbers)
GET /v1/admin/models · POST /models/promote	{ model_version_id, comparison? }	versions, calibration, metrics, which is active
GET · POST /v1/admin/users · POST /api-keys	–	accounts and simulator keys
GET /v1/admin/audit · /audit/verify	object_type?, object_id?	the permanent record, and whether its chain is intact
GET /v1/metrics/overview	hours	signal_frequency, latency, queue, alert_budget

Everyone with case access — Search

ENDPOINT	SEND	FIELDS THE SCREEN READS
GET /v1/transactions/search	q (reference or name fragment), channel?, auth_result?, risk_level?, min_amount_minor?, limit, offset	matching payments with their decision and any alert
GET /v1/alerts	since_id, limit	id, case_id, risk_level, score_0_100, raised_at, transaction_ref, amount_minor, channel, instrument, rail, auth_result, display_name

What this means for Paul

- Build the menu and every button from `me.permissions`; never from the role name.
- Treat the worklist as the source of truth: refetch on each stream "alert" event and after every action; do not patch the list by hand.
- Show money from `_minor` fields only, divided by 100 at the last step.
- Handle 409 on "take next case" by fetching again; handle 403 by explaining what the role cannot do, not by showing an error.
- Every risk level, clock and chart state carries a word, not just a colour, using the validated colours in the Desk design.
- The administrator's screens never link to a case, even by ID.

Gaps between the full dashboard and today's API

#	DASHBOARD NEEDS	TODAY	PROPOSED	OWNER
1	Ops lead: how often analysts follow the advice	saved with each outcome, not returned	operations → advice_followed { rate, by_action[] }	Chidera
2	Admin: "who catches what" and the calibration chart	held in the model's evaluation file; shape not fixed in the API	GET /v1/admin/models/{id}/evaluation → per_type[], calibration_bins[]	Chidera
3	InfoSec: their queue without filtering in the browser	escalated_to on items, no filter	GET /v1/worklist?escalated_to=INFOSEC	Chidera
4	InfoSec: dormant accounts visible in the list	only inside case detail	worklist item → dormant_days	Chidera
5	Admin: rule fired vs turned into an alert	signal_frequency has n only	signal_frequency[] → on_alerts	Chidera
6	Ops lead: alerts per hour against capacity	available (alert_volume + alert_budget)	– chart built on Operations	Paul DONE

Changing this contract

- A field changes in the same commit as this document and the contract test that checks it (Chidera, task 5 in *Chidera's Deliverables*).
- A new need goes in the gaps table with an owner before anyone builds around it.
- The contract freezes on the Monday of week five, with the features.

Sources: backend/riskradar/api/routers/, api/schemas.py, security/rbac.py, worker/scoring.py, frontend/src/lib/api.js and useStream.js, as of 11 September 2026. Companions: Chidera's Deliverables, Risk Radar Blueprint, Risk Radar Desk.